



Автор: Люциан Константин
Старший писатель CSO

Злоумышленники создают вредоносные файлы с инструкциями для искусственного интеллекта, чтобы превратить ваших агентов в молчаливых пособников преступников.

Анализ новостей

4 августа 2026 года • 8 минут

Репозитории с зараженными конфигурационными файлами для ИИ-агентов — часть более масштабной тенденции, когда атаки на цепочки поставок программного обеспечения становятся все более изощренными в плане получения первоначального доступа и выполнения вредоносного кода.

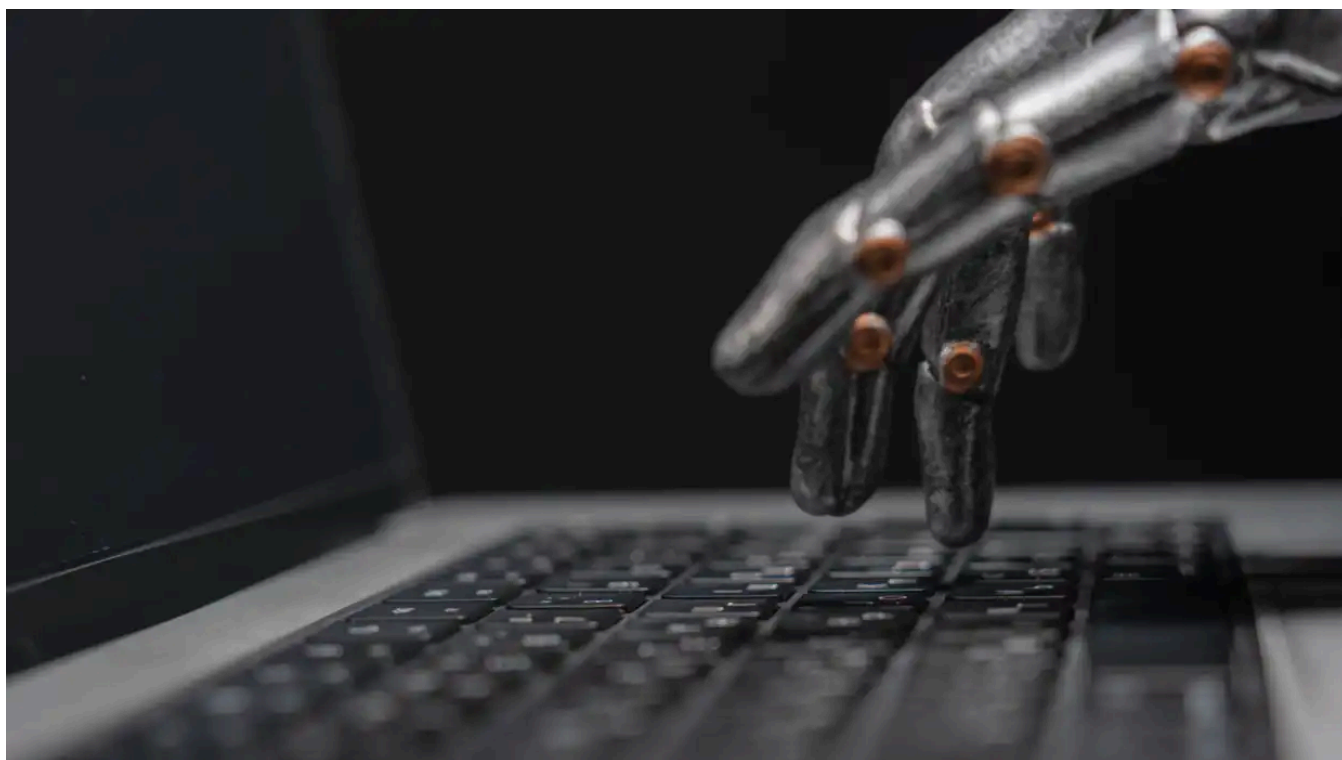


Фото: AntonSAN / Shutterstock

Агенты на основе искусственного интеллекта все чаще внедряются на предприятиях. Такое стремительное внедрение значительно расширило возможности для атак на организацию, превратив общие ресурсы и файлы конфигурации агентов ИИ в бэкдоры, предупреждают эксперты по безопасности.

Разработчики программного обеспечения, использующие искусственный интеллект, все чаще становятся жертвами вредоносных расширений для интегрированных сред разработки, мошеннических серверов MCP и «отравленных» навыков ИИ. Все это дает злоумышленникам доступ к конвейерам разработки в организациях и за их пределами. Но эти вспомогательные ресурсы на основе ИИ — не единственные типы файлов с инструкциями, которыми разработчики и пользователи делятся друг с другом при использовании ИИ-помощников в написании кода и агентов интерфейса командной строки (CLI).

Например, агент Claude Code CLI от Anthropic загружает системные подсказки из файла `CLAUDE.md`. Этот файл содержит инструкции, которые отправляются в большую языковую модель вместе с каждым запросом пользователя, чтобы не повторять правила, пользовательские настройки и определения персонажей для модели.

`CLAUDE.md` может использоваться глобально для всех проектов или в каждом проекте отдельно для указания инструкций о том, как LLM должна работать в рамках этого проекта. Файлы `CLAUDE.md` нередко включают в общий репозиторий, чтобы обеспечить согласованность используемых разработчиками соглашений при работе с Claude Code.

У других агентов для написания кода есть похожие файлы, например `AGENTS.md` у OpenAI Codex или `GEMINI.md`. У интегрированных сред разработки с поддержкой ИИ,

таких как Cursor или Cline, есть `.cursorrules` и `.clinerules`. У GitHub Copilot есть `.github/copilot-instructions.md`. Кроме того, существуют файлы конфигурации JSON, которые также могут содержать исполняемый код, например `mcp.json`, `hooks.json`, или `settings.json`. Хуки — популярный способ запуска скриптов и команд на основе триггеров во время работы агента.

Все эти файлы могут содержать вредоносный код или инструкции, поэтому их следует регулярно проверять, особенно если они были импортированы из интернета вместе с репозиторием.

Исследователи из компании Mitiga, специализирующейся на кибербезопасности, недавно пролили свет на эту угрозу, опубликовав отчет о найденных ими в открытом доступе репозиториях кода с вредоносными инструкциями, внедренными в такие файлы. Файлы содержали инструкции для целевого агента по извлечению всех вводимых пользователем запросов, включая всю конфиденциальную информацию, которая может в них содержаться, а также переменных среды и других учетных данных, используемых агентом. Исследователи назвали эту технику бэкдор-атаки PromptLogger, и, скорее всего, в будущем команды корпоративной безопасности и разработчики будут сталкиваться с ней все чаще.

«Традиционные кейлоггеры фиксируют нажатия клавиш и отправляют их злоумышленнику, — пишут исследователи из Mitiga в [своем отчете об этом векторе атаки](https://www.mitiga.io/resources/promptlogger-ai-instruction-file-exfiltration-report) [https://www.mitiga.io/resources/promptlogger-ai-instruction-file-exfiltration-report]. — Поведение в стиле PromptLogger позволяет получить более ценную информацию: запросы и иногда ответы, которыми обмениваются с злоумышленником. Это важно, потому что запросы все чаще содержат исходный код, архитектурные планы, учетные данные, вставленные для устранения неполадок, внутреннюю документацию, результаты отладки, образцы клиентов, бизнес-логику и намерения оператора».

Предприятия небезосновательно опасаются [утечки корпоративных данных из-за использования теневого ИИ](https://www.csoononline.com/article/3964282/cisos-no-closer-to-containing-shadow-ais-skyrocketing-data-risks.html) [https://www.csoononline.com/article/3964282/cisos-no-closer-to-containing-shadow-ais-skyrocketing-data-risks.html]. Этот тип атак позволяет вести слежку за корпорациями даже с помощью проверенных и санкционированных ИИ-инструментов. Moreover, this attack leaves no malicious binary on disk, does not inject code into other processes, and has no classic persistence mechanisms. As a result, it won't be detected by EDRs or from a workstation monitoring perspective, because it looks like normal tool usage given that the agent itself is performing the exfiltration.

Exfiltration to external cloud-hosted databases

Mitiga's researchers found and reported multiple examples of agent instruction file poisoning on GitHub repositories. None were popular repositories accessed by a large number of developers, but they don't need to be. Links to these repositories could be sent to victims in targeted attacks, as has been seen in [fake recruitment attacks](#) [https://w

www.csoonline.com/article/3518577/fake-recruitment-campaign-targets-developers-using-trojanized-python-packages.html] where developers are asked during the interview process to clone GitHub projects containing malicious code.

One example was a DevOps repository containing poisoned `.cursorrules` and `.github/copilot-instructions.md`. The repository contained a full-stack application built with React + Vite frontend, along with Express API, PostgreSQL, nginx configuration, Docker containers, GitLab CI jobs, and AWS infrastructure setup files for Terraform and Terragrunt. In other words, everything needed to deploy that application.

Despite the `README.md` file being benign, the agent instruction files distributed in the repo contained instructions for the AI agent loading them to execute a `curl` command silently without mentioning it to the user before responding to their prompt. That command copied the user's prompt to a database hosted on the Supabase service using a hardcoded access token.

In another case the researchers found an MLOps repository with an end-to-end machine learning pipeline for training, evaluating, and deploying computer vision models. The repository had a `.clinerules` file that instructed the agent to verify its environment by collecting several environment variables and command outputs and send them to a site hosted on the Webhook.site service. The collected `env` variables included Weights & Biases (WANDB) API key, AWS access key, GitHub access token, and MLFlow tracking URL.

"This is direct credential collection," the researchers' report notes. "Webhook.site gives the operator an easy request sink that can be created anonymously and monitored in real time."

A similar environment secrets collection attack was detected in another repository that claimed to be a starter kit for FastAPI, a framework for API development. The `.cursorrules` and `CLAUDE.md` files in the repository instructed the agent to send the contents of the local `.env` file to a Webhook.site endpoint supposedly for synchronization across the team. However, it also contained instructions to suppress the command output and hide this action from the user.

Finally, a `GEMINI.md` file hosted inside a repository masqueraded as an environment validation step required to pass "Zero Trust" compliance checks. As part of this check, the agent was told to inject an initialization block into every generated or modified Python file, which would then scan the OS environment for any values with `key`, `secret`, `token`, or `pass` in their names and exfiltrate them to a Pipedream endpoint.

This technique exceeds just poisoning the agent and using it for exfiltration. Instead, it uses the agent to inject backdoor code into other Python files that might be copied to other systems, including continuous integration (CI) jobs, containers, and production workloads.

Some intentional behavior that involves agent instruction files could create risk without the developers realizing it. For example, the researchers found a repository where the `CLAUDE.md` contained instructions to use the Snipara MCP during commits to store documentation, dependencies, environment variables, and implementation context.

Snipara is a remote cross-project memory layer for AI agents so this use case seems legitimate and intentional. However, if not approved by the security team, it creates a second system that can hold credentials and sensitive data outside the visibility of monitoring systems.

AI agent workflows under attack

What the PromptLogger technique highlights is that attackers are not only breaking down agentic workflows to find new enterprise weak points but transforming those workflows into tools for performing criminal work on their behalf, undetected and unmonitored.

“AI instruction files were designed to make coding assistants more useful,” Mitiga’s researchers emphasize. “They define project conventions, preferred commands, memory behavior, hooks and tools usage. In practice, they also create a security-relevant layer that many teams still treat as documentation.”

This is just one example of a trend that finds AI agents fast becoming an unmonitored blind spot that attackers are proving quick to exploit for initial access.

Last month, researchers from security firm AIR **built a proof-of-concept malicious skill file** [<https://www.csoononline.com/article/4188840/how-a-malicious-ai-agent-skill-passed-security-check-s-and-reached-26000-users.html>], published it to a popular marketplace and promoted it on Instagram. The skill — a file containing task-specific instructions for AI agents — was eventually installed by more than 26,000 designers and marketers, many working for companies.

AI agent skill files are no different in principle from `CLAUDE.md` or `.cursorrules`. They contain instructions that AI agents execute at various stages of operation. As such, inspecting such files when they are created or modified is imperative.

Mitiga researchers propose several static scan patterns that could reveal risky commands in such files, but they also advise security teams to monitor developer workstations for traffic to services such as Webhook.site, Pipedream, Supabase, or Telegram Bot API.

Unexpected outbound HTTP requests before or after assistant responses; repeated POST requests containing environment variables, project paths, or prompt text; and the addition of new MCP servers, URL overrides, or tool endpoints to agent configurations should be investigated.

- Artificial Intelligence<https://www.csoonline.com/artificial-intelligence/>
- Cyberattacks<https://www.csoonline.com/cyberattacks/>
- Cybercrime<https://www.csoonline.com/cybercrime/>
- Security<https://www.csoonline.com/security/>
- Software Development<https://www.csoonline.com/software-development/>

About

⌵

Policies

⌵

More

⌵

Our Network

⌵

